



SESIÓN 8 Paginación + Ordenación + Filtros (Spring Data JPA)

"Nunca devuelvas listas infinitas."



Bloque 1 — Problema real

Ahora mismo, GET /api/books devuelve todos los libros.

Problemas:

- En producción puede haber miles → lento
- Consume memoria y red
- No permite buscar ni ordenar

 **Objetivo:**

- Paginar resultados (page, size)
- Ordenar (sort=title,asc)
- Filtrar (por category, author, title contains)
- Mantener DTOs + arquitectura (Controller → Service → Repository)



Mensaje clave: "Una API profesional pagina por defecto."



Bloque 2 — Teoría esencial

1. Pageable y Page

Pageable = lo que pide el cliente (page, size, sort)

Page = respuesta paginada (content + metadata)

2. Ordenación

Spring soporta query param estándar:

?sort=title,asc

?sort=author,desc

3. Filtros

Dos niveles:

Nivel 1 (sencillo, sesión 8): métodos derivados en Repository

Nivel 2 (pro, más adelante): Specification (Criteria API)

Hoy hacemos nivel 1.



Bloque 3 — Estructura del proyecto

```
com.cladsb1.books
├── controller
│   └── BookController
├── service
│   └── BookService
├── repository
│   ├── BookRepository
│   └── CategoryRepository
├── entity
│   ├── Book
│   └── Category
├── dto
│   ├── request
│   │   └── BookRequestDTO
│   └── response
│       ├── BookResponseDTO
│       └── PageResponseDTO
```



Bloque 4 — DTO de respuesta paginada

dto/response/PageResponseDTO.java

```
package com.cladsb1.books.dto.response;

import java.util.List;

/**
 * Respuesta paginada estable para el cliente.
 * Evita exponer directamente Page (que es de Spring y puede cambiar).
 */
public record PageResponseDTO<T>(
    List<T> content,
    int pageNumber,
    int pageSize,
    long totalElements,
    int totalPages,
    boolean last
) {}
```



Mensaje clave: "El contrato de paginación también es parte de la API."



Bloque 5 — Repository: métodos paginados con filtros

repository/BookRepository.java

```
package com.cladsb1.books.repository;

import com.cladsb1.books.entity.Book;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;

public interface BookRepository extends JpaRepository {

    // Filtrar por author (exacto, ignorando mayúsculas)
    Page findByAuthorIgnoreCase(String author, Pageable pageable);

    // Filtrar por título parcial (contains), ignorando mayúsculas
    Page findByTitleContainingIgnoreCase(String title, Pageable pageable);

    // Filtrar por nombre de categoría (atravesando relación)
    Page findByCategory_NameIgnoreCase(String categoryName, Pageable pageable);

}
```



Aquí se ve el poder de Spring Data: Category_Name navega la relación.

Bloque 6A — Service con paginación y filtros (CON lambdas)

 service/BookService.java (añade estos métodos)

```
package com.cladsb1.books.service;

import com.cladsb1.books.dto.response.BookResponseDTO;
import com.cladsb1.books.dto.response.PageResponseDTO;
import com.cladsb1.books.entity.Book;
import com.cladsb1.books.repository.BookRepository;
import org.springframework.data.domain.*;
import org.springframework.stereotype.Service;

@Service
public class BookService {

    private final BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public PageResponseDTO<BookResponseDTO> findPaged(String author, String title,
                                                    String category, Pageable pageable) {
        Page<BookDb> page = bookRepository.findAll(pageable);

        if (author != null && !author.isBlank()) {
            page = bookRepository.findByAuthorIgnoreCase(author, pageable);
        } else if (title != null && !title.isBlank()) {
            page = bookRepository.findByTitleContainingIgnoreCase(title, pageable);
        } else if (category != null && !category.isBlank()) {
            page = bookRepository.findByCategory_NameIgnoreCase(category, pageable);
        } else {
            page = bookRepository.findAll(pageable);
        }

        // Mapping con streams/lambdas
        return new PageResponseDTO<> (
            page.getContent().stream().map(this::toDTO).toList(),
            page.getNumber(),
            page.getSize(),
            page.getTotalElements(),
            page.getTotalPages(),
            page.isLast()
        );
    }

    private BookResponseDTO toDTO(Book b) {
        return new BookResponseDTO(
            b.getId(),
            b.getTitle(),
            b.getAuthor(),
            b.getCategory().getName()
        );
    }
}
```

Nota didáctica: hoy hacemos filtros "prioritarios" (author o title o category). Combinar filtros múltiples lo dejamos para "nivel 2: Specification".



Bloque 6B — Service sin streams

```
public PageResponseDTO findPaged(String author, String title,
                                String category, Pageable pageable) {

    Page page;

    if (author != null && !author.isBlank()) {
        page = bookRepository.findByAuthorIgnoreCase(author, pageable);
    } else if (title != null && !title.isBlank()) {
        page = bookRepository.findByTitleContainingIgnoreCase(title, pageable);
    } else if (category != null && !category.isBlank()) {
        page = bookRepository.findByCategory_NameIgnoreCase(category, pageable);
    } else {
        page = bookRepository.findAll(pageable);
    }

    // Mapping SIN streams
    List content = new ArrayList<>();
    for (Book b : page.getContent()) {
        content.add(toDTO(b));
    }

    return new PageResponseDTO<>(
        content,
        page.getNumber(),
        page.getSize(),
        page.getTotalElements(),
        page.getTotalPages(),
        page.isLast()
    );

}
```

(El controller es igual; no necesita lambdas.)

Bloque 7A — Controller con Pageable automático (CON lambdas)

 controller/BookController.java (nuevo GET)

```
package com.cladsb1.books.controller;

import com.cladsb1.books.dto.response.BookResponseDTO;
import com.cladsb1.books.dto.response.PageResponseDTO;
import com.cladsb1.books.service.BookService;
import org.springframework.data.domain.Pageable;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookService bookService;

    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    /**
     * Paginación + filtros:
     * /api/books?page=0&size=5&sort=title,asc
     * /api/books?author=Asimov&page=0&size=10
     * /api/books?category=programming&page=1&size=3
     */
    @GetMapping
    public ResponseEntity<PageResponseDTO<BookResponseDTO>> getBooks(
        @RequestParam(required = false) String author,
        @RequestParam(required = false) String title,
        @RequestParam(required = false) String category,
        @ParameterObject Pageable pageable
    ) {
        return ResponseEntity.ok(bookService.findPaged(author, title, category, pageable));
    }
}
```



Bloque 8 — Tabla de comportamiento

Request	Qué hace	Resultado
/api/books?page=0&size=5	pagina	200 + PageResponseDTO
/api/books?page=1&size=5	siguiente página	200
/api/books?sort=title,asc	ordena	200
/api/books? author=Asimov&page=0&size=10	filtra author	200
/api/books? title=clean&page=0&size=10	contains title	200
/api/books? category=programming&page=0&size=10	filtra relación	200



Bloque 9 — Pruebas en Swagger

Pruébalos en Swagger así:

- /api/books?page=0&size=3
- /api/books?page=1&size=3
- /api/books?sort=title,asc&page=0&size=5
- /api/books?category=programming&page=0&size=5
- /api/books?title=code&page=0&size=5



Tip: mete 10–15 libros de prueba antes.

Bloque 10 — Errores típicos

Errores comunes

- devolver List enorme (sin paginación)
- exponer Page directamente (contrato acoplado a Spring)
- filtros combinados a base de if infinitos (para eso hay Specifications)
- size gigantes (en producción se limita)

Bloque 11 — Cierre

 **Mensaje clave:**

"Nunca devuelvas listas infinitas."